

Tutoriel HTTP/Express Node.js

- [Tutoriel HTTP/Express Node.js](#)
 - [Partie 1 : serveur HTTP natif Node.js](#)
 - [Installation](#)
 - [Servir différents types de contenus](#)
 - [Mode développement](#)
 - [Gestion manuelle des routes](#)
 - [Partie 2 : framework Express](#)
 - [Création du serveur](#)
 - [Ajout de middlewares](#)
 - [Rendu avec EJS](#)
 - [Gestion d'erreurs](#)
 - [Conclusion](#)

Ce tutorial est inspiré de [How To Create a Web Server in Node.js with the HTTP Module](#) et compléter avec une partie sur Express.

Ce tutorial vous fait prendre en main l'environnement Node.js avec un petit projet de serveur web monté pas à pas, utilisant essentiellement les bibliothèques standards de Node.js. Le framework <http://expressjs.com/> sera introduit ensuite.

- installer Node.js <https://nodejs.org/en/download/> pour votre environnement
- créer un dossier `devweb-tp5`

RENDU vous devrez remplir le fichier `README.md` avec les questions du sujet et **commiter/pousser sur GitHub**. Les différentes étapes à réaliser seront aussi committées.

Partie 1 : serveur HTTP natif Node.js

Installation

Exécuter la commande `npm init` dans le dossier `devweb-tp5`. Répondre avec les valeurs par défaut, sauf pour *entry point*: (`index.js`) où donner la valeur `server-http.mjs`. À ce stade, un fichier `package.json` a du être créé à peu près comme suit.

```
{  
  "name": "devweb-tp5",
```

```

"version": "1.0.0",
"description": "",
"main": "server-http.mjs",
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1"
},
"author": "",
"license": "ISC"
}

```

Toujours dans le dossier `devweb-tp5`, créer le fichier `server-http.mjs` avec le contenu suivant :

```

import http from "node:http";

const host = "localhost";
const port = 8000;

function requestListener(_request, response) {
  response.writeHead(200);
  response.end("<html><h1>My first server!</h1></html>");
}

const server = http.createServer(requestListener);
server.listen(port, host, () => {
  console.log(`Server is running on http://${host}:${port}`);
});

```

Enfin, exécuter la commande `node server-http.mjs` et vérifier que votre application web fonctionne en vous connectant avec votre navigateur.

Question 1.1 donner la liste des en-têtes de la réponse HTTP du serveur.

Servir différents types de contenus

Maintenant, remplacer la fonction `requestListener()` par la suivante et tester :

```

function requestListener(_request, response) {
  response.setHeader("Content-Type", "application/json");
  response.end(JSON.stringify({ message: "I'm OK" }));
}

```

Question 1.2 donner la liste des en-têtes qui ont changé depuis la version précédente.

Remplacer enfin la fonction `requestListener()` par la suivante et tester :

```

import fs from "node:fs/promises";

```

```
function requestListener(_request, response) {
  fs.readFile("index.html", "utf8")
    .then((contents) => {
      response.setHeader("Content-Type", "text/html");
      response.writeHead(200);
      return response.end(contents);
    })
    .catch((error) => console.error(error));
}
```

Question 1.3 que contient la réponse reçue par le client ?

Question 1.4 quelle est l'erreur affichée dans la console ? Retrouver sur <https://nodejs.org/api> le code d'erreur affiché.

Modifier la fonction `requestListener()` précédente pour que le client reçoive une erreur 500 si `index.html` est introuvable en remplaçant le callback de la méthode `Promise.catch()`.

Maintenant, renommer le fichier `__index.html` en `index.html` et tester à nouveau.

Enfin, reprenez `requestListener()` dans le style `async/await`.

Question 1.5 donner le code de `requestListener()` modifié *avec gestion d'erreur* en `async/await`.

Commit/push dans votre dépôt Git.

Mode développement

Dans le dossier `devweb-tp5` exécuter les commandes suivantes :

- `npm install cross-env --save`
- `npm install nodemon --save-dev`

Question 1.6 indiquer ce que cette commande a modifié dans votre projet.

Ensuite, remplacer la propriété `"scripts"` du fichier `package.json` par la suivante :

```
"scripts": {
  "test": "echo \"Error: no test specified\" && exit 1",
  "http-dev": "cross-env NODE_ENV=development nodemon server-http.mjs",
  "http-prod": "cross-env NODE_ENV=production node server-http.mjs"
},
```

Exécuter `npm run http-dev`, visiter le site, puis *pendant que le serveur s'exécute* modifier le fichier `server-http.mjs` en ajoutant la ligne `console.log("NODE_ENV =", process.env.NODE_ENV);`.

Enregistrer le fichier et vérifier qu'il y a eu rechargement automatique grâce à

<https://nodemon.io/>. Ensuite, faire la même chose avec la commande `npm run http-prod`.

Question 1.7 quelles sont les différences entre les scripts `http-dev` et `http-prod` ?

Les fichiers `eslint.config.mjs` et `.prettierrc` sont fournis dans le dossier `TP5/base`. Exécuter la commande suivante pour installer les dépendances :

```
npm i -D prettier eslint @eslint/js @eslint/eslintrc eslint-plugin-n eslint-plugin-import
```

Vérifier que l'autoformatage avec <https://prettier.io> et le linting avec <https://eslint.org/> fonctionnent dans VSCode et en ligne de commande avec les commandes suivantes :

- `npx eslint server-http.mjs`
- `npx prettier server-http.mjs --write`

Commit/push dans votre dépôt Git.

Gestion manuelle des routes

Remplacer la fonction `requestListener()` par la suivante :

```
async function requestListener(request, response) {
  response.setHeader("Content-Type", "text/html");
  try {
    const contents = await fs.readFile("index.html", "utf8");
    switch (request.url) {
      case "/index.html":
        response.writeHead(200);
        return response.end(contents);
      case "/random.html":
        response.writeHead(200);
        return response.end(`<html><p>${Math.floor(100 * Math.random())}</p></html>`);
      default:
        response.writeHead(404);
        return response.end(`<html><p>404: NOT FOUND</p></html>`);
    }
  } catch (error) {
    console.error(error);
    response.writeHead(500);
    return response.end(`<html><p>500: INTERNAL SERVER ERROR</p></html>`);
  }
}
```

Tester les routes suivantes :

- `http://localhost:8000/index.html`
- `http://localhost:8000/random.html`
- `http://localhost:8000/`

- `http://localhost:8000/dont-exist`

Question 1.8 donner les codes HTTP reçus par votre navigateur pour chacune des quatre pages précédentes.

Maintenant, on veut ajouter une route `/random/:nb` où `:nb` est un paramètre entier avec le nombre d'entiers à générer. Ajouter cette route au `switch` et reprendre la page `random.html` pour générer autant de nombres qu'indiqué dans l'URL.

Pour cela, utiliser `request.url.split("/")` qui va décomposer le chemin demandé et faire le `switch` sur le premier niveau de l'arborescence. Faites en sorte que le `switch` traite `/index.html` et `/` de la même façon.

Commit/push dans votre dépôt Git.

Partie 2 : framework Express

On voit que la gestion manuelle des routes avec un grand `switch` va devenir complexe et laborieuse. Les frameworks serveur comme <http://expressjs.com/>, <https://koajs.com/>, <https://www.fastify.io/> ou <https://hapi.dev/> vont s'occuper de cette plomberie et proposer une API pour enregistrer des *handlers* aux différentes routes de l'application.

Création du serveur

Créer le fichier `server-express.mjs` et exécuter la commande suivante :

```
npm install --save express http-errors loglevel morgan
```

Question 2.1 donner les URL des documentations de chacun des modules installés par la commande précédente.

Ensuite, sur le modèle des scripts `http-prod` et `http-dev` du fichier `package.json`, créer les scripts `express-prod` et `express-dev`.

Ensuite, ajouter le contenu suivant au fichier `server-express.mjs`

```
import express from "express";
import morgan from "morgan";

const host = "localhost";
const port = 8000;

const app = express();

app.get(["/", "/index.html"], async function (request, response, next) {
  response.sendFile("index.html", { root: "." });
});
```

```
});

app.get("/random/:nb", async function (request, response, next) {
  const length = request.params.nb;
  const contents = Array.from({ length })
    .map((_) => `<li>${Math.floor(100 * Math.random())}</li>`)
    .join("\n");
  return response.send(`<html><ul>${contents}</ul></html>`);
});

app.listen(port, host);
```

Question 2.2 vérifier que les trois routes fonctionnent.

Question 2.3 lister les en-têtes des réponses fournies par Express. Lesquelles sont nouvelles par rapport au serveur HTTP ?

Remplacer la dernière ligne de `server-express.mjs` par les suivantes

```
const server = app.listen(port, host);

server.on("listening", () =>
  console.info(
    `HTTP listening on http://${server.address().address}:${server.address().port} with mc
  ),
);

console.info(`File ${import.meta.url} executed.`);
```

Question 2.4 quand l'événement `listening` est-il déclenché ?

Commit/push dans votre dépôt Git.

Ajout de middlewares

Ici, la route de la page d'accueil charge dynamiquement à chaque requête une ressource statique. Ce n'est pas très performant, d'autant plus qu'un *middleware* Express existe déjà pour ça.

- créer un sous-dossier `static`
- déplacer le fichier `index.html` dans le sous-dossier `static`
- extraire l'élément `<style>` de `index.html` dans un nouveau fichier `style.css` que vous lierez à `index.html` avec `<link rel="stylesheet" href="style.css">`
- remplacer la route de la page d'accueil par `app.use(express.static("static"));`

Question 2.5 indiquer quelle est l'option (activée par défaut) qui redirige `/` vers `/index.html` ?

Question 2.6 visiter la page d'accueil puis rafraichir (Ctrl+R) et *ensuite* **forcer** le rafraichissement (Ctrl+Shift+R). Quels sont les codes HTTP sur le fichier `style.css` ? Justifier.

Ajouter la ligne `if (app.get("env") === "development") app.use(morgan("dev"));` au bon endroit dans `server-express.mjs` pour activer le middleware Morgan.

Commit/push dans votre dépôt Git.

Rendu avec EJS

Le moteur de templating <https://ejs.co/> est l'équivalent de Jinja utilisé pour Python/Flask dans l'écosystème Nodes.js/Express. Une [extension VSCode](#) est disponible pour EJS.

On va utiliser le moteur EJS pour améliorer la page `random` générée dynamiquement côté serveur.

1. créer un sous-dossier `views/` et créer un fichier `views/random.ejs` avec le contenu ci-après;
2. exécuter la commande `npm install --save ejs` ;
3. ajouter la ligne `app.set("view engine", "ejs");` à `server-express.mjs` ;
4. modifier le *handler* de la route `/random/:nb` avec `response.render("random", {numbers, welcome});` pour appeler le moteur de rendu, où `numbers` est un tableau de nombres aléatoires (comme précédemment) et `welcome` une chaîne de caractères.

Contenu de `views/random.ejs`

```
<!DOCTYPE html>
<html>
  <head>
    <meta charset="utf-8" />
    <meta name="viewport" content="width=device-width, initial-scale=1.0" />
    <link rel="stylesheet" type="text/css" href="https://cdn.jsdelivr.net/npm/normalize.css" />
    <link rel="stylesheet" href="/style.css" />
    <title>Tutorial</title>
  </head>

  <body>
    <div class="center">
      <h1><%= welcome %></h1>
      <% numbers.forEach(element => { %>
        <code><%= element %></code>
      <% }) %>
    </div>
  </body>
</html>
```

Commit/push dans votre dépôt Git.

Gestion d'erreurs

On va maintenant vérifier que le paramètre `/random/:nb` est bien un nombre. Si ce n'est pas le cas, il faut retourner une erreur HTTP 400. Pour cela, utiliser le module <https://github.com/jshttp/http-errors>

1. ajouter le module `http-errors` avec `npm`
2. ajouter le `import ... from ...` correspondant dans `server-express.mjs`
3. dans la route `/random/:nb`, faites la vérification avec `const length = Number.parseInt(request.params.nb, 10);` puis `Number.isNaN(length)`, si le paramètre, n'est pas un nombre, produire une erreur 400 avec `next(createError(400));`

Commit/push dans votre dépôt Git.

Avec cette solution, l'erreur n'est pas bien rendue sur le client car elle passe dans le **handler d'erreur par défaut d'Express**. De plus, quand on visite une page qui n'existe pas, par exemple `http://localhost:8000/javascript`, la 404 n'est pas terrible non plus.

Ajouter, *tout à la fin des routes*, juste avant `app.listen(port, host);`, deux nouveaux *handlers* comme suit :

```
app.use((request, response, next) => {
  console.debug(`default route handler : ${request.url}`);
  return next(createError(404));
});

app.use((error, _request, response, _next) => {
  console.debug(`default error handler: ${error}`);
  const status = error.status ?? 500;
  const stack = app.get("env") === "development" ? error.stack : "";
  const result = { code: status, message: error.message, stack };
  return response.render("error", result);
});
```

Ensuite, créer, sur le modèle de `random.ejs`, une vue `error.ejs` dont le corps est comme suit :

```
<body>
  <div class="center">
    <h1>Error <%= code %></h1>
    <p><%= message %></p>
  </div>
  <% if (stack != null) { %>
    <pre><%= stack %></pre>
  <% } %>
</body>
```

Question 2.7 vérifier que l’affichage change bien entre le mode *production* et le mode *development*.

Commit/push dans votre dépôt Git.

Enfin, chargez le module `loglevel` avec `import logger from "loglevel";` puis fixer un niveau de verbosité avec `logger.setLevel(logger.levels.DEBUG);` .

Remplacez tous les `console.log()` et variantes par `logger.error()` , `logger.warn()` , `logger.info()` , `logger.debug()` ou `logger.trace()` approprié.

Modifier le niveau de verbosité, par exemple `logger.setLevel(logger.levels.WARN);` et vérifier l’affichage.

Commit/push dans votre dépôt Git.

Conclusion

À ce stade du tutoriel, vous avez vues les principales étapes de la création d’une application Node.js/Express. Ces étapes seront déjà réalisées dans le projet de départ du [TP6](#).